

ITERATION 1

Test Case 1.1: Feature Branch Deployment by Developer 1

- **Description:** Developer 1 deploys changes from a feature branch to DEV.
- **SQL Files:**
- `create_table_customers.sql`

```
CREATE TABLE DEV.CUSTOMERS (  
    CUSTOMER_ID INT,  
    NAME STRING,  
    EMAIL STRING  
);
```

- **Steps:**
- Developer 1 pushes feature branch `feature/add_customers`.
- Pipeline triggers feature branch deployment to DEV.
- **Expected Results:**
- Table `CUSTOMERS` created in DEV.
- Logs show successful execution.
- **Validation:**

```
DESCRIBE TABLE DEV.CUSTOMERS;
```

Test Case 1.2: DEV Deployment (Incremental since last PROD)

- **Description:** Identify all SQL changes since last PROD deployment and deploy to DEV.
- **Steps:**
- Compare `dev-master` against last PROD deployment metadata.
- Execute incremental SQL files (e.g., `create_table_customers.sql`).
- **Expected Results:**
- DEV updated with all changes.
- Deployment logs reflect only incremental SQLs.
- **Validation:**

```
SELECT * FROM DEV.CUSTOMERS;
```

Returns 0 rows initially.

Test Case 1.3: PROD Deployment

- **Description:** Deploy metadata-tracked SQLs from DEV to PROD.
 - **Steps:**
 - Take metadata of deployable SQL files executed in DEV.
 - Deploy to PROD.
 - **Expected Results:**
 - PROD updated with `CUSTOMERS` table.
 - Deployment logs confirm successful execution.
-

ITERATION 2

Test Case 2.1: Multiple Feature Branch Commits by Developers 1 & 2 (DDL Changes)

- **Description:** Multiple commits in separate feature branches with DDL changes; potential overlapping object modifications.
- **SQL Files:**
 - Developer 1:
 - `alter_table_customers_add_phone.sql`

```
ALTER TABLE DEV.CUSTOMERS ADD COLUMN PHONE STRING;
```
 - `alter_table_customers_add_address.sql`

```
ALTER TABLE DEV.CUSTOMERS ADD COLUMN ADDRESS STRING;
```
 - Developer 2:
 - `alter_table_customers_add_dob.sql`

```
ALTER TABLE DEV.CUSTOMERS ADD COLUMN DOB DATE;
```
- **Steps:**
 - Developers 1 & 2 push feature branches.
 - Raise pull requests.
 - Resolve conflicts and merge into `master`.
- **Expected Results:**
 - DEV schema includes all added columns: `PHONE`, `ADDRESS`, `DOB`.
 - Metadata updated for deployable SQL files.

Test Case 2.2: DEV Deployment (Incremental DDL since last PROD)

- **Description:** Deploy all incremental DDL changes since last PROD deployment (Iteration 1).
- **Steps:**
 - Compare merged `master` with last PROD metadata.

- Execute incremental SQL files in DEV (`ALTER TABLE` statements).
- **Expected Results:**
- Table `CUSTOMERS` now has `PHONE`, `ADDRESS`, and `DOB` columns.
- Pipeline logs confirm only incremental DDLs were executed.
- **Validation:**

```
DESCRIBE TABLE DEV.CUSTOMERS;
```

Test Case 2.3: PROD Deployment

- **Description:** Deploy incremental DDLs from DEV to PROD using metadata.
- **Steps:**
- Collect metadata of SQL files executed in DEV.
- Deploy sequentially to PROD.
- **Expected Results:**
- PROD schema updated with all new columns.
- Deployment logs reflect only incremental DDLs deployed.
- **Validation:**

```
DESCRIBE TABLE PROD.CUSTOMERS;
```

Columns: `CUSTOMER_ID`, `NAME`, `EMAIL`, `PHONE`, `ADDRESS`, `DOB`

SECTION: Validation Queries for DEV & PROD

```
DESCRIBE TABLE CUSTOMERS;
```

- **Expected Result in PROD after Iteration 2:** | Column Name | Data Type | |-----|-----| |
`CUSTOMER_ID` | INT | | `NAME` | STRING | | `EMAIL` | STRING | | `PHONE` | STRING | | `ADDRESS` |
STRING | | `DOB` | DATE |

Key Checks Across Iterations

- Metadata tracking for incremental deployments is accurate.
- DEV deployments always reflect all DDL changes since last PROD.
- Feature branch deployments are isolated and validated end-to-end.
- Conflicts are resolved before merging into `master`.
- Only necessary incremental DDLs are deployed to DEV and PROD.